SIMATS ENGINEERING

Department of Computer Science and Engineering

Assessment Tool 2 - Case Study Answers

Course Code / Title: CSA06 / Design and Analysis of Algorithms

CO Assessed: CO3 - Articulation (BL4, BL5)

Topic: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication

1. Introduction to Divide and Conquer

The **Divide and Conquer** paradigm is a foundational algorithm design strategy that involves breaking down a complex problem into smaller, more manageable sub-problems of the same type, solving these sub-problems recursively, and finally combining their solutions to form the solution to the original problem. This approach is highly effective in optimizing time complexity and forms the basis of many efficient algorithms in computer science.

The methodology operates in three distinct steps:

- **Divide:** Partition the given problem into smaller sub-problems. This step generally takes $O(1)$ or $O(n)$ time.
- **Conquer:** Recursively solve the sub-problems. If the sub-problem sizes are small enough (base case), solve them directly.
- **Combine:** Merge the solutions of the sub-problems to generate the final output. The efficiency of the algorithm often heavily depends on the combination step.

2. Merge Sort

Merge Sort is a quintessential example of the Divide and Conquer strategy. It works by recursively halving the array until each sub-array contains a single element. Since a single element is inherently sorted, the algorithm then repeatedly merges the sub-arrays to produce new sorted sub-arrays until a single sorted array remains.

2.1 Algorithm Analysis

The performance of Merge Sort is highly consistent. Regardless of the initial order of elements (best, worst, or average case), the array is always divided into two halves, and the merge process takes linear time. The recurrence relation is:

$$T(n) = 2T(n/2) + O(n)$$

Using the Master Theorem for divide-and-conquer recurrences, this yields a time complexity of $O(n \log n)$. However, Merge Sort requires $O(n)$ auxiliary space, which is a drawback compared to in-place sorting algorithms.

2.2 C Implementation

```c
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

3. Quick Sort

Quick Sort is another highly efficient, comparative sorting algorithm based on Divide and Conquer. Unlike Merge Sort, which does the heavy lifting during the combination phase, Quick Sort performs its primary work during the division (partitioning) phase.

3.1 Algorithm Analysis

The algorithm selects a 'pivot' element and partitions the array around the pivot such that elements smaller than the pivot are placed to its left, and elements greater are placed to its right. The recursive calls are then applied to the sub-arrays.

- **Best and Average Case:** When the partition always divides the array into two nearly equal halves, the recurrence relation is $T(n) = 2T(n/2) + O(n)$, leading to $O(n \log n)$ time complexity.
- **Worst Case:** Occurs when the array is already sorted (or reversely sorted) and the smallest or largest element is chosen as the pivot. The recurrence is $T(n) = T(n-1) + O(n)$, yielding $O(n^2)$.

3.2 C Implementation

```
#include <stdio.h>

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

4. Binary Search

Binary Search is a classic search algorithm that leverages the Divide and Conquer strategy to find the position of a target value within a sorted array. It is highly efficient because it halves the search space during each step.

4.1 Algorithm Analysis

The algorithm compares the target value to the middle element of the array. If they are unequal, the half in which the target cannot lie is eliminated, and the search continues on the remaining half. The recurrence relation is:

$$T(n) = T(n/2) + O(1)$$

This resolves to a worst-case time complexity of $O(\log n)$. The space complexity is $O(1)$ for the iterative implementation and $O(\log n)$ for the recursive implementation due to the call stack.

4.2 C Implementation

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int x) {
    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Check if x is present at mid
        if (arr[mid] == x)
            return mid;

        // If x greater, ignore left half
        if (arr[mid] < x)
            low = mid + 1;

        // If x is smaller, ignore right half
        else
            high = mid - 1;
    }
    return -1; // Element not present
}
```

5. Matrix Multiplication (Divide & Conquer and Strassen's)

The standard method of matrix multiplication for two $n \times n$ matrices requires $O(n^3)$ operations. The Divide and Conquer approach aims to reduce this computational complexity.

5.1 Simple Divide and Conquer

A matrix can be partitioned into four $n/2 \times n/2$ sub-matrices. The product of two such matrices can be expressed as 8 multiplications of these sub-matrices, followed by 4 additions. The recurrence relation is:

$$T(n) = 8T(n/2) + O(n^2)$$

Using the Master Theorem, this still results in $O(n^3)$ complexity. Therefore, the basic divide and conquer approach does not provide a theoretical improvement over the naive method.

5.2 Strassen's Algorithm

Volker Strassen introduced an innovative modification to the Divide and Conquer strategy for matrix multiplication. By defining a specific set of 7 matrix multiplications (instead of 8), he reduced the recursive calls.

The recurrence relation for Strassen's algorithm is:

$$T(n) = 7T(n/2) + O(n^2)$$

This subtle reduction from 8 to 7 recursive calls drastically changes the asymptotic behavior, yielding a time complexity of approximately $O(n^{2.81})$. This is a significant optimization for large matrices, though standard algorithms may still be faster for smaller matrices due to the higher constant factors and memory overhead inherent in Strassen's approach.

5.3 Python Implementation snippet (Strassen Concept)

```
import numpy as np

def strassen_multiply(A, B):
    n = len(A)
    # Base case
    if n == 1:
        return A * B

    mid = n // 2
    A11, A12 = A[:mid, :mid], A[:mid, mid:]
    A21, A22 = A[mid:, :mid], A[mid:, mid:]
    B11, B12 = B[:mid, :mid], B[:mid, mid:]
    B21, B22 = B[mid:, :mid], B[mid:, mid:]

    # 7 Products computed by Strassen
    M1 = strassen_multiply(A11 + A22, B11 + B22)
    M2 = strassen_multiply(A21 + A22, B11)
    M3 = strassen_multiply(A11, B12 - B22)
    M4 = strassen_multiply(A22, B21 - B11)
    M5 = strassen_multiply(A11 + A12, B22)
    M6 = strassen_multiply(A21 - A11, B11 + B12)
    M7 = strassen_multiply(A12 - A22, B21 + B22)

    # Recombine
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6

    # Combine into single matrix C
    C = np.vstack((np.hstack((C11, C12)), np.hstack((C21, C22))))
    return C
```

6. Case Study Conclusion & Final Remarks

The algorithms discussed herein—Merge Sort, Quick Sort, Binary Search, and Strassen's Matrix Multiplication—demonstrate the profound impact of the Divide and Conquer strategy on computational efficiency. By systematically breaking down large instances into manageable subsets, processing them efficiently, and recombining the results, we overcome the limitations of brute-force approaches.

Key takeaways include:

- **Scalability:** Divide and Conquer directly targets time complexity optimization, making algorithms scalable for enormous datasets (e.g., Quick Sort in modern databases).
- **Mathematical Rigor:** The performance characteristics can be accurately modelled and predicted using recurrence relations and the Master Theorem.
- **Trade-offs:** Improvements in time complexity often come at the expense of space complexity (e.g., auxiliary arrays in Merge Sort, call stack depth in Quick Sort). Architectural context should guide algorithm selection in real-world engineering tasks.

This comprehensive analysis addresses the core requirements of the CSA06 Case Study, substantiating the theoretical discussion with standard implementations.